

4. Encapsulation

- Objectif
- Déclaration avec @property
- Utilisations des méthodes getter, setter et deleter
- Propriétés en lecture seule
- Propriétés calculées
- Validation des valeurs

Objectif

Les **propriétés** offrent un moyen de contrôler l'accès aux attributs d'une classe. Elles nous permettent de définir des méthodes spéciales, telles que les **getters**, les **setters** et les **deleters**, qui seront automatiquement appelées lors de l'accès, de la modification ou de la suppression de ces attributs.

Grâce à l'utilisation de **propriétés**, il est possible de garantir l'encapsulation des données, ce qui signifie que les détails internes de la classe restent cachés à l'extérieur et ne peuvent être modifiés que de manière contrôlée via des méthodes spécifiques



Déclaration avec @property

La décoration `@property` nous permet de créer des attributs qui peuvent être accédés comme des attributs de classe, sans nécessiter l'utilisation de parenthèses.

Cette fonctionnalité permet de définir des propriétés directement dans la classe, facilitant ainsi l'accès et la gestion des données de manière transparente.

Par exemple, dans notre classe `Voiture`, nous pouvons définir une méthode `infos` décorée avec `@property`, qui permet d'accéder aux informations sur la voiture, telles que la marque, le modèle, la couleur et la vitesse actuelle, comme s'il s'agissait d'attributs de classe.

```
1 class Voiture:
2     marque = "Toyota"
3     modele = "Corolla"
4     couleur = "Rouge"
5     vitesse_actuelle = 0
6
7     @property
8     def infos(self):
9         return f"Marque: {self.marque}, ect..."
10
11 # Création d'une instance de la classe Voiture
12 ma_voiture = Voiture()
13
14 # Accès aux informations de la voiture
15 print("Informations de la voiture:")
16 print(ma_voiture.infos)
```

Utilisation des méthodes getter, setter et deleter

L'utilisation des méthodes **getter**, **setter** et **deleter** est une pratique essentielle en programmation orientée objet pour contrôler l'accès aux attributs d'une classe. Chaque méthode possède un rôle spécifique :

- **Getter (Accesseur)** : Cette méthode permet d'accéder à la valeur d'un attribut sans permettre de la modifier directement. Elle garantit un accès sécurisé et contrôlé aux données de la classe.
- **Setter (Mutateur)** : Le setter est utilisé pour modifier la valeur d'un attribut en appliquant des règles de validation ou de transformation. Il offre un moyen de modifier les données tout en maintenant l'intégrité de la classe.
- **Deleter (Suppresseur)** : Le deleter est responsable de la suppression d'un attribut de la classe. Il permet de libérer les ressources associées à l'attribut et de maintenir la cohérence de l'objet.

Utilisation des méthodes : getter

Un **getter** est une méthode utilisée pour accéder à la valeur d'un attribut d'une classe.

Celui-ci est implémenté à l'aide du décorateur **@property**, ce qui permet d'accéder à l'attribut comme s'il s'agissait d'un attribut de classe directement, sans avoir à utiliser de parenthèses pour appeler la méthode.

Ainsi, lorsque nous accédons à l'attribut `marque` de la classe `Voiture` à travers `ma_voiture.marque`, Python invoque automatiquement la méthode **getter** associée à cet attribut, permettant de récupérer sa valeur.

```
1 class Voiture:
2     _marque = "Toyota"
3
4     # Utilisation de @property pour définir un getter pour l'attribut 'marque'
5     @property
6     def marque(self):
7         """
8         Méthode getter pour l'attribut '_marque'.
9         Elle permet d'accéder à la valeur de l'attribut '_marque'.
10        """
11        return self._marque
12
13    # Utilisation de @property.getter pour définir un getter pour l'attribut 'marque'
14    @property.getter
15    def marque(self):
16        """
17        Méthode getter pour l'attribut '_marque'.
18        Elle permet également d'accéder à la valeur de l'attribut '_marque'.
19        """
20        return self._marque
21
22
23    # Création d'une instance de la classe Voiture
24    ma_voiture = Voiture()
25
26    # Accès à la marque (lecture seule)
27    print("Marque de la voiture :", ma_voiture.marque)
```

Utilisation des méthodes : setter

Un **setter** est une méthode utilisée pour modifier la valeur d'un attribut d'une classe.

Celui-ci est implémenté à l'aide du décorateur **@property.setter**. Lorsque nous attribuons une nouvelle valeur à un attribut à travers le setter, comme dans `ma_voiture.marque = "Nissan"`, Python invoque automatiquement la méthode setter associée à cet attribut, permettant de mettre à jour sa valeur.

Le setter permet de contrôler l'affectation des valeurs aux attributs, ce qui permet de valider ou de transformer les données avant de les assigner.

```
1 class Voiture:
2     _marque = "Toyota"
3
4     @property
5     def marque(self):
6         return self._marque
7
8     @marque.setter
9     def marque(self, nouvelle_marque):
10        """
11        Méthode setter pour l'attribut '_marque'.
12        Elle permet de modifier la valeur de l'attribut '_marque'.
13        """
14        print("Modification de la marque en", nouvelle_marque)
15        self._marque = nouvelle_marque
16
17
18 # Création d'une instance de la classe Voiture
19 ma_voiture = Voiture()
20
21 # Affichage de la marque initiale
22 print("Marque initiale de la voiture :", ma_voiture.marque)
23
24 # Modification de la marque à l'aide du setter
25 ma_voiture.marque = "Nissan"
26
27 # Affichage de la nouvelle marque après modification
28 print("Nouvelle marque de la voiture :", ma_voiture.marque)
```

Utilisation des méthodes : delete

Le **delete** est une fonctionnalité essentielle pour assurer la gestion complète des attributs d'une classe.

Il permet de définir un comportement spécifique lors de la suppression d'un attribut, offrant ainsi un contrôle supplémentaire sur la manipulation des données.

L'utilité du **delete** réside dans sa capacité à garantir la cohérence et l'intégrité des données de la classe, en permettant par exemple de libérer des ressources associées à un attribut lors de sa suppression.

```
1 class Voiture:
2     _marque = "Toyota"
3
4     @property
5     def marque(self):
6         return self._marque
7
8     @marque.delete
9     def marque(self):
10        """
11        Méthode delete pour l'attribut '_marque'.
12        Elle permet de supprimer la valeur de l'attribut '_marque'.
13        """
14        print("Suppression de la marque")
15        del self._marque
16
17
18 # Création d'une instance de la classe Voiture
19 ma_voiture = Voiture()
20
21 # Affichage de la marque initiale
22 print("Marque initiale de la voiture :", ma_voiture.marque)
23
24 # Suppression de la marque à l'aide du delete
25 del ma_voiture.marque
26
27 # Affichage de la marque après suppression (devrait générer une erreur)
28 print("Marque de la voiture après suppression :", ma_voiture.marque)
```

Propriétés en lecture seule

Les propriétés en lecture seule sont des attributs dont la valeur ne peut être modifiée une fois initialisée. Elles sont définies en utilisant uniquement une méthode getter avec le décorateur `@property`, ce qui empêche toute modification directe de la valeur de la propriété.

Par exemple, dans une classe `Voiture`, nous pouvons définir une propriété en lecture seule pour l'identifiant de la voiture, garantissant ainsi que cet attribut ne peut être modifié après sa définition initiale ou via une méthode interne à la classe.

```
1 class Voiture:
2     _identifiant = "ABC123" # Attribut défini dans la classe
3
4     @property
5     def identifiant(self):
6         return self._identifiant
7
8 # Création d'une instance de la classe Voiture
9 ma_voiture = Voiture()
10
11 # Accès à la propriété en lecture seule
12 print("Identifiant de la voiture :", ma_voiture.identifiant)
13
14 # Tentative de modification de l'identifiant (ne fonctionnera pas)
15 ma_voiture.identifiant = "XYZ789" # générera une erreur
```


Propriétés calculées

Les propriétés calculées offrent la possibilité de définir des attributs dont la valeur est déterminée dynamiquement à partir d'autres attributs de l'objet.

Contrairement aux propriétés simples dont la valeur est stockée directement, les propriétés calculées sont évaluées au moment de l'accès à la propriété.

Cela permet de définir des comportements personnalisés pour l'accès aux données, en permettant de réaliser des calculs ou des transformations sur les données existantes.

```
1 class Voiture:
2     _vitesse = 0
3     _temps_ecoule = 0
4
5     @property
6     def vitesse(self):
7         return self._vitesse
8
9     @vitesse.setter
10    def vitesse(self, nouvelle_vitesse):
11        self._vitesse = nouvelle_vitesse
12
13    @property
14    def temps_ecoule(self):
15        return self._temps_ecoule
16
17    @temps_ecoule.setter
18    def temps_ecoule(self, nouveau_temps):
19        self._temps_ecoule = nouveau_temps
20
21    @property
22    def distance_parcourue(self):
23        # Calcul de la distance parcourue en fonction de la vitesse et du temps écoulé
24        return self.vitesse * self.temps_ecoule
25
26    # Création d'une instance de la classe Voiture
27    ma_voiture = Voiture()
28
29    # Attribution d'une vitesse et d'un temps écoulé
30    ma_voiture.vitesse = 80 # km/h
31    ma_voiture.temps_ecoule = 2 # heures
32
33    # Accès à la propriété calculée "distance_parcourue"
34    print("Distance parcourue par la voiture :", ma_voiture.distance_parcourue, "kilomètres")
```

Validation des valeurs

La validation des valeurs des propriétés est importante pour garantir l'intégrité des données. Les méthodes setter permettent de contrôler les valeurs avant de les attribuer à une propriété, assurant ainsi la conformité aux règles métier.

Dans notre exemple avec la classe `Voiture`, nous avons défini un validateur pour la propriété `vitesse`. Ce validateur garantit que la vitesse est positive. En cas de valeur invalide, une exception sera levée.

```
1 class Voiture:
2     def __init__(self):
3         self._vitesse = 0
4
5     @property
6     def vitesse(self):
7         return self._vitesse
8
9     @vitesse.setter
10    def vitesse(self, nouvelle_vitesse):
11        if nouvelle_vitesse < 0:
12            raise ValueError("La vitesse ne peut pas être négative.")
13        self._vitesse = nouvelle_vitesse
14
15    ma_voiture = Voiture()
16
17    # Modification de la vitesse
18    try:
19        ma_voiture.vitesse = 120 # km/h
20        print("Vitesse de la voiture :", ma_voiture.vitesse)
21    except ValueError as e:
22        print("Erreur :", e)
```

Exercices

1. Mettre en place des propriétés sur les différents attributs des classes.

Elephant :

- nom (lecture et écriture)
- rassasier (sur 100) (lecture seule)
- bonheur (sur 100) (lecture seule)
- en_vie (lecture seule)
- soigneur (lecture et écriture)

Soigneur :

- nom (lecture et écriture)
- date_naissance (lecture seule)
- expérience (lecture seule)
- nombre_animaux_responsable (lecture seule)

Enclos :

- nom (lecture et écriture)
- capacite_max (lecture et écriture)
- taille (lecture seule)
- liste_animaux (lecture seule)

Exercices

2. Mettre en place une propriété calculée qui va donner l'âge du soigneur sur base de sa date de naissance.
3. Réalisez des tests pour vérifier si votre programme continue à tourner correctement, le cas échéant appliquer les correctifs.